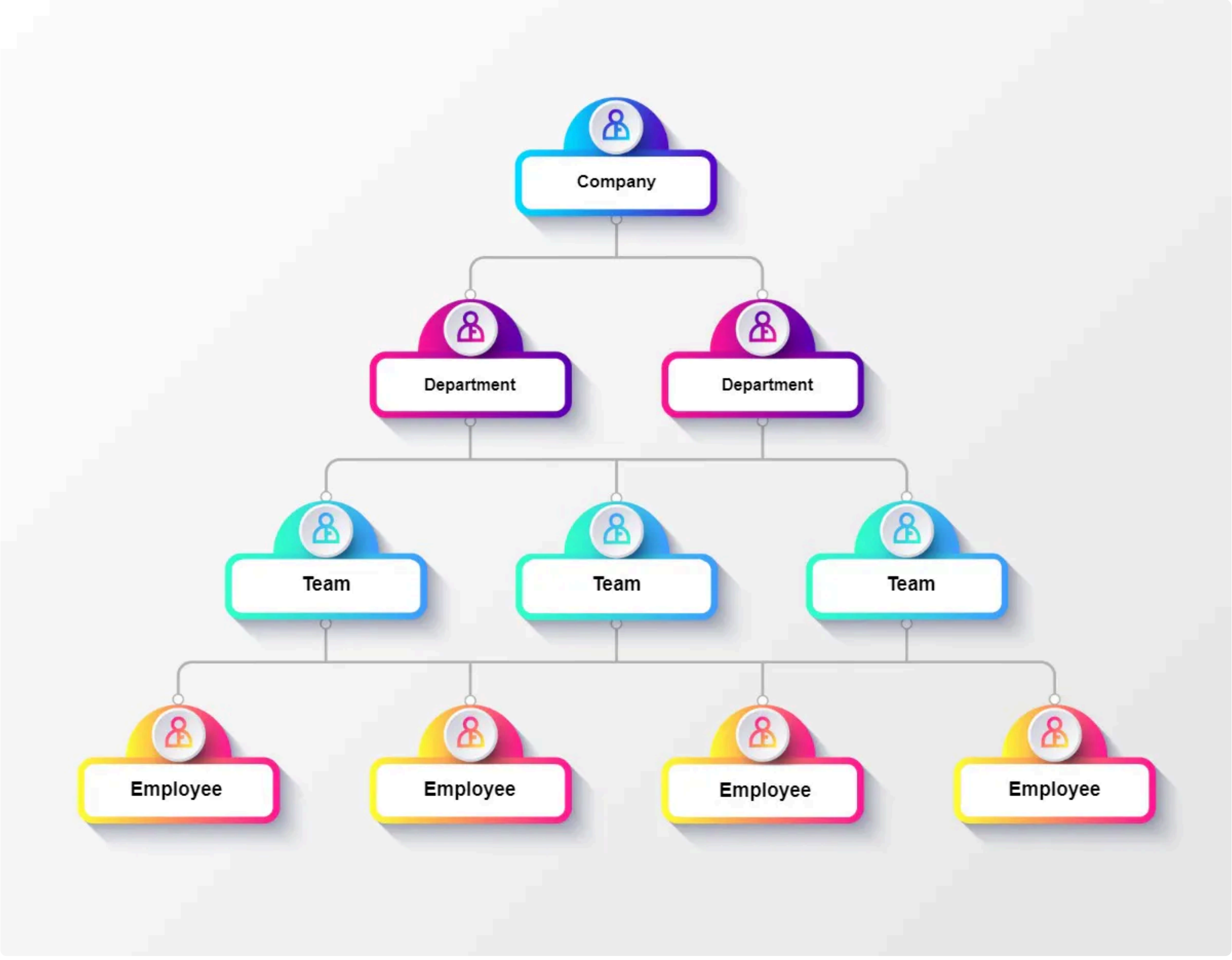
Composite Pattern

The Composite Pattern is a structural design pattern that enables the composition of objects into tree structures to represent part-whole hierarchies. This pattern allows clients to treat individual objects and object compositions uniformly.

Let us look at a problem that we come across through an illustration image



Composite Pattern - Problem

The illustration shows the organizational structure of a company in a simplified manner while drawing attention to an issue that the Composite Pattern can effectively resolve. The organization is organized into departments within this structure, and teams within each department are composed of workers.

This configuration shows how difficult it can be to aggregate and manage data across hierarchies, for example, to determine the total number of hours worked by an employee. Such a hierarchical structure can be difficult to manage without the Composite Pattern, particularly when it comes to tasks that must be completed consistently at all levels (departments, teams, and employees).

But what is the solution? Are you curious to know the solution? Let's dive in!

Using the Composite Pattern for the organizational structure problem, you would interact with both Departments and Employees through a common interface that declares a method for calculating the total working hours.

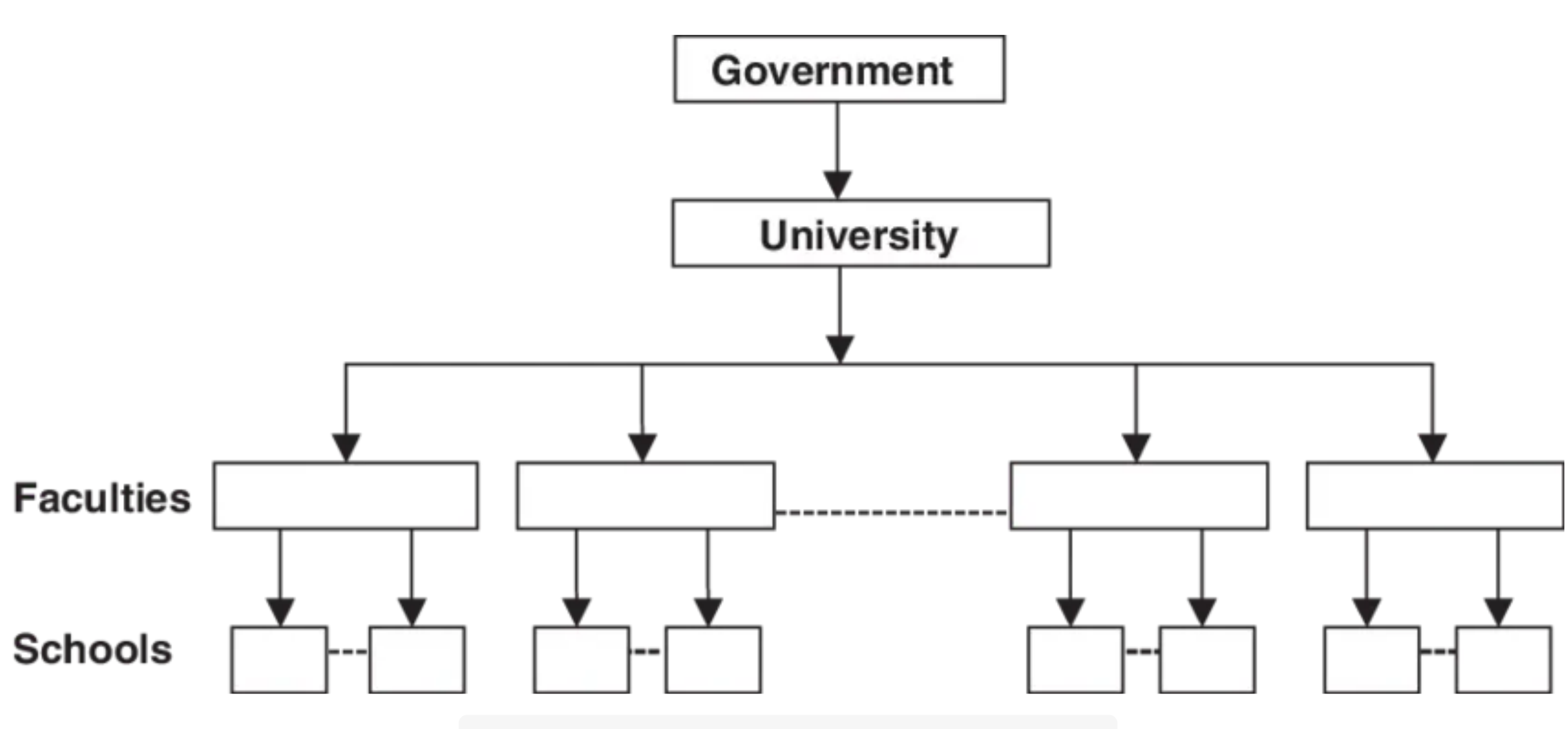
How does this method function?

- For an individual employee, it would simply return the employee's hours.
- For a department, it would iterate over each member the department contains, accumulate their hours, and return the total for that department.
- If a member happens to be a sub-department, this sub-department would in turn aggregate the hours of its members, and this process would continue recursively until the hours of all members are accounted for.
- A department could also account for additional factors, such as overtime hours.

Real-world Example

An academic university's structure serves as a real-world analogy for the Composite Pattern.

The University itself is at the top of the hierarchy. The University is organized into a number of composite Schools or Faculties, such as the School of Engineering, School of Medicine, and School of Humanities. These schools have departments, such as the mechanical engineering and computer science departments, which are composites that can hold more individuals or structures. A department is made up of programs (such as the Master's Program in Robotics or the Undergraduate Program in Computer Science), and each program is composed of courses. A professor or lecturer (the leaves in the structure) teaches each course.



Composite Pattern - Read-world Example

All schools are subject to university-level policies and decisions, which are then implemented by the schools according to the needs of their departments, programs, and even the way that courses are taught.

The core of the Composite Pattern is demonstrated by this academic structure, which allows the university to uniformly manage operations at any level, from broad strategic planning to the intricacies of individual course offerings.

Structure of Composite Pattern

The composite pattern has some of the key components:

- **Component**:

  This is an abstract class or interface that specifies the shared functions for leaf and composite nodes. It outlines guidelines for managing children and might even set the default behavior for them.

- **Leaf**:

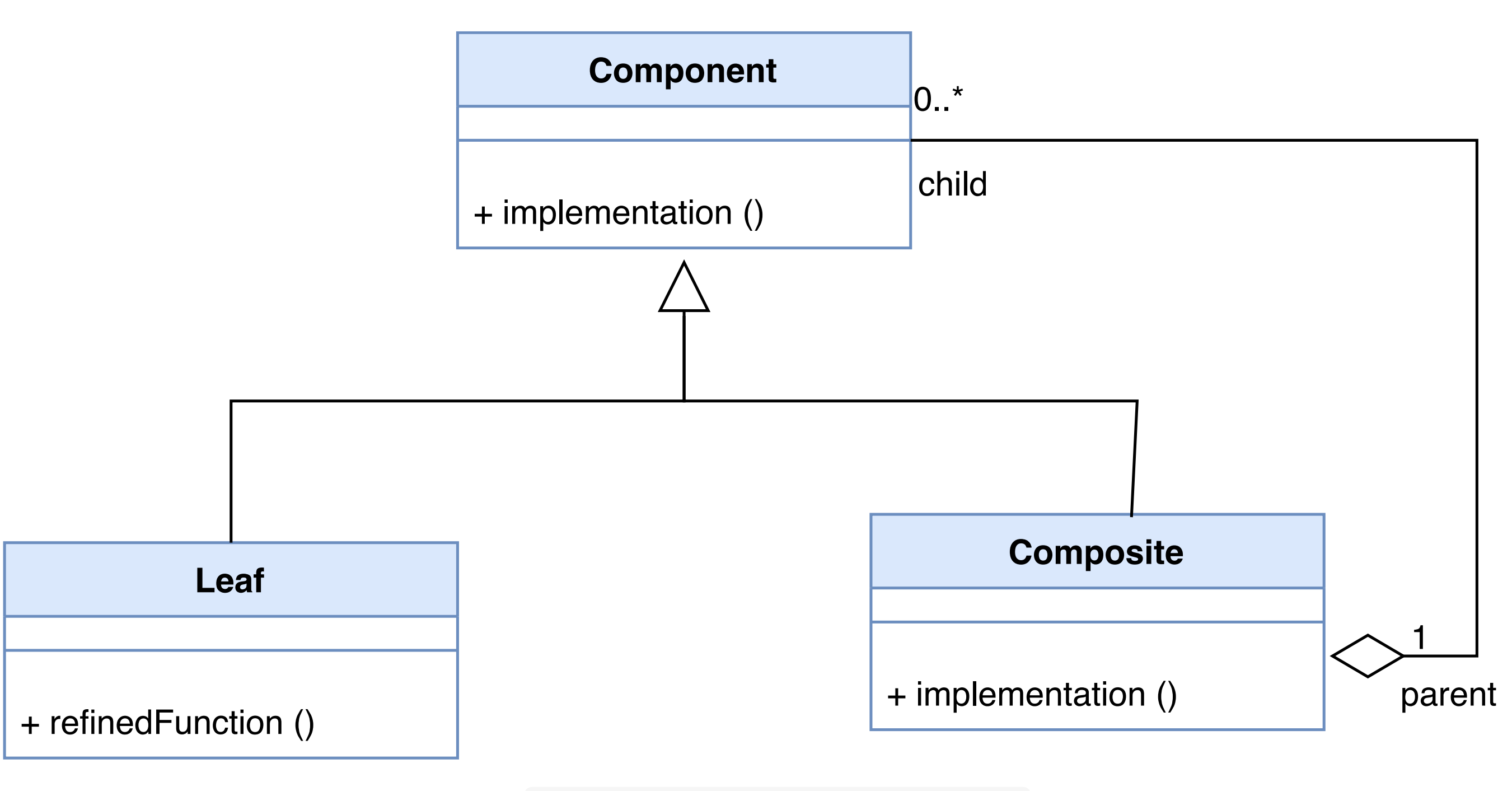
  These are the fundamental components that make up the structure. Things made of leaves don't have children. They carry out the component's primitive behavior.

- **Composite**:

  A composite object has other composite objects and leaf elements. It implements methods to add, remove, or access child components that are specified in the component interface.

- **Client**:

  Using the component interface, the client can modify objects within the composition.



Composite Pattern - Class Diagram

Implementation of Composite Pattern

Let's implement the organization example in multiple languages.

Pseudocode:

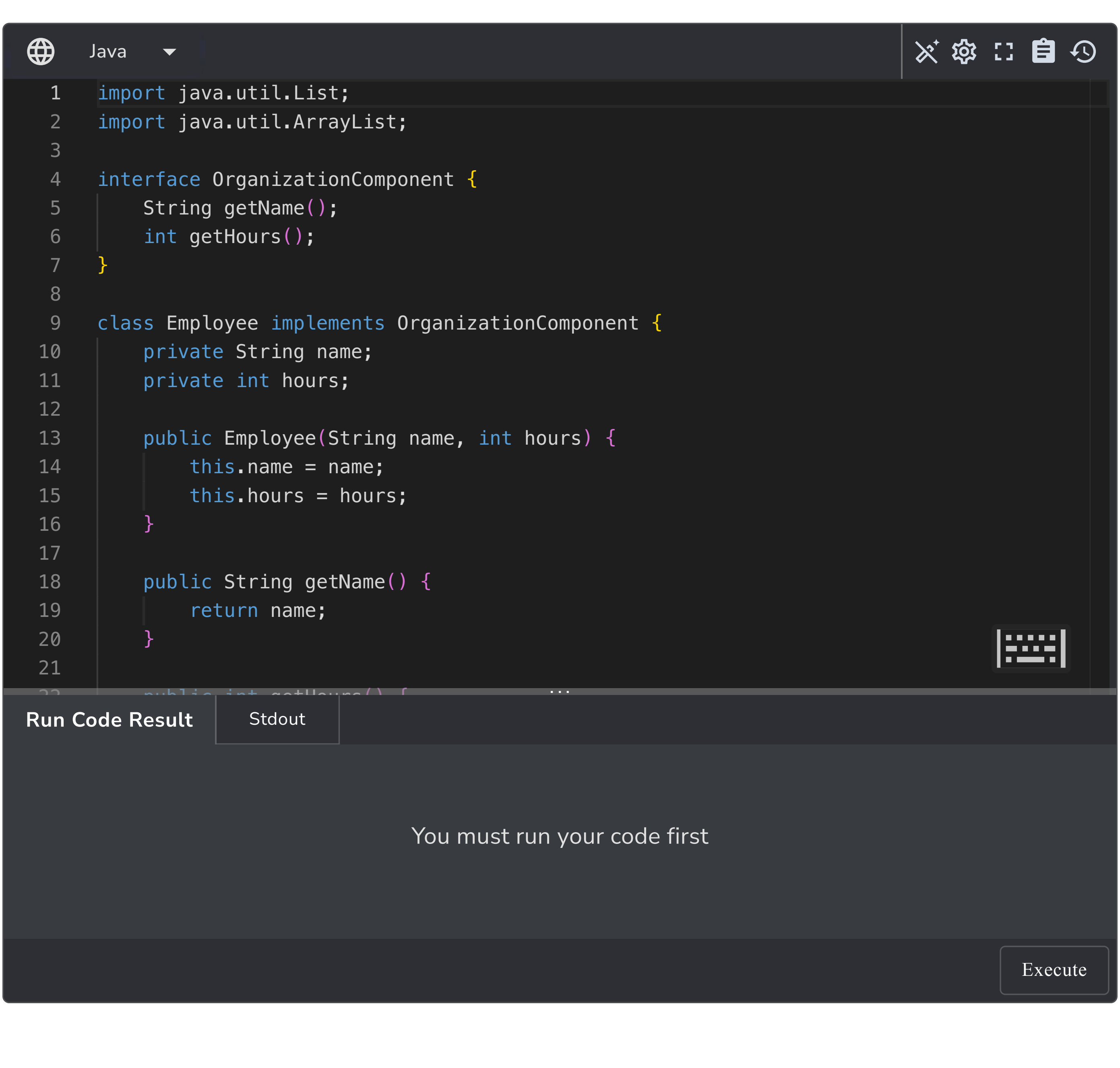
Here's the pseudocode for the Composite Pattern applied to the employee problem:

```
INTERFACE OrganizationComponent
    METHOD getName()
    METHOD getHours()

CLASS Employee IMPLEMENTS OrganizationComponent
    PRIVATE name, hours
    CONSTRUCTOR(name, hours)
        this.name = name
        this.hours = hours
    METHOD getName()
        RETURN name
    METHOD getHours()
        RETURN hours

CLASS Department IMPLEMENTS OrganizationComponent
    PRIVATE name, components = new List<OrganizationComponent>
    CONSTRUCTOR(name)
        this.name = name
    METHOD getName()
        RETURN name
    METHOD getHours()
        totalHours = 0
        FOR component IN components
            totalHours += component.getHours()
        RETURN totalHours
    METHOD addComponent(component)
        components.add(component)
```

In this pseudocode, the `OrganizationComponent` serves as the common interface for both `Employee` (leaf) and `Department` (composite), allowing clients to interact with them uniformly. A `Department` can have other `Department(s)` or `Employee(s)` as children and calculate total hours by aggregating the hours of its components.

Here's how you can implement the organizational structure example using the Composite Pattern across different languages:

Implementation



```java
import java.util.List;
import java.util.ArrayList;

interface OrganizationComponent {
    String getName();
    int getHours();
}

class Employee implements OrganizationComponent {
    private String name;
    private int hours;

    public Employee(String name, int hours) {
        this.name = name;
        this.hours = hours;
    }

    public String getName() {
        return name;
    }

```

You must run your code first

Application of Composite Pattern

The Composite Pattern is useful in many situations where you have a part-whole hierarchy and you want to handle individual objects and their compositions consistently. Typical uses for these include:

- **Graphical User Interface (GUIs)**

  Some of the GUIs have complex layouts. Composite patterns can handle such GUIs very efficiently. Panels, frames, and buttons are examples of components that can be nested in different ways. This pattern enables the same treatment of individual parts and compositions (such as a panel with buttons).

- **File Directories**

  Directories in file systems can contain files and other directories. The Composite Pattern enables operations such as searching, copying, and size calculation to be performed uniformly on both files (leaf nodes) and directories (composite nodes).

- **Organizational Structure**

  This pattern can be used to manage organizational charts that include departments, teams, and individual employees, as demonstrated in the example. This enables the execution of tasks such as computing total work hours or uniformly allocating resources throughout the hierarchy.

- **Menus**

  Another appropriate use case is application menus, particularly those with nested submenus. Whether it's a last action item or a container for additional items, every menu item (which may also be a submenu) is handled consistently.

- **Document Object Model (DOM) of web pages**

  The DOM is a tree structure of elements used in web development. The Composite Pattern can be used to apply styling, scripting, and other effects consistently to individual elements and nested structures (such as a div that contains paragraphs and other divs).

Pros and Cons of Composite Pattern

Here's a table summarizing the pros and cons of the Composite Pattern:

Pros	Cons
Simplifies Client Code: It makes client code simpler as it allows the management of individual objects and compositions uniformly.	**Overgeneralization**: Can result in an overly generic design. Certain operations may be more appropriate for leaf components than composite ones.
Ease of Adding New Kinds of Components: Adding new components to the existing code is easy since the code remains unchanged.	**Increased Complexity**: Adds complexity by requiring all components to share a common interface, which may not be necessary for simple structures.
Clear Structure: Gives objects a clear tree-like hierarchy, which is advantageous for part-whole relationships.	**Potential for Too Much Freedom**: Might permit clients to carry out actions on specific components that are illogical, possibly resulting in misuse or errors.
Flexibility in Design: makes it easier to construct complex structures out of smaller components.	**Indirect Overhead**: Operations on components might incur additional overhead due to indirection and safety checks.
Reusable Components: promotes component reusability because it allows components to be used separately or in a composite.	

This table demonstrates the many benefits that the Composite Pattern offers when handling complex structures, including increased design flexibility and simpler client code. But, it's important to be aware of any possible drawbacks, such as overgeneralization and added complexity, particularly in situations where the part-whole hierarchy may not be strictly required.